Pipeline Run V2 — CellRanger v2 (condition_1 / condition_2)

Single-Cell RNA-Seq Analysis of Arabidopsis thaliana

SingleCell Pipeline
2026-07-07

Materials and Methods

Materials

The pipeline source code is available at `https://github.com/clifford2001/SingleCell`:

```
git clone https://github.com/clifford2001/SingleCell.git
cd SingleCell
```

The repository is meant to sit inside a working directory that also holds the raw data and the pipeline's output folder — for instance `~/your-workdir/SingleCell`, with `~/your-workdir/resultados` alongside it. That parent directory, not the repository alone, is what gets mounted into the container, so that code, input matrices, and results are all visible together at `/workspace`.

Docker packages R, Python, and every pipeline dependency into one portable **image** (`matigara/scrnaseq:latest`), so the analysis runs identically on any machine with Docker installed. Two equivalent ways to start it:

Option 1 — single command

```
docker run -it --rm \
  -v ~/your-workdir:/workspace \
  matigara/scrnaseq:latest /bin/bash
```

Pulls the image if missing and opens a shell. Replace `~/your-workdir` with the actual path to that working directory if it lives somewhere else.

Option 2 — the repository's `docker-compose.yml`

Run as-is from the repository root:

```
docker compose pull
docker compose run --rm r bash
```

`pull` only reads that file and downloads the image it names — unrelated to git, which just placed the file on disk. Same `~/your-workdir` → `/workspace` mount as Option 1.

Either way, `/workspace` **is** `~/your-workdir` on the host: files written there persist after the container exits.

Once inside the shell, Chapter 1-2 commands are typed into R, and Chapter 3 commands into Python:

```
R
```

```
python3
```

Computational requirements

The pipeline was validated on two Linux servers: CellRanger alignment ran on a 128-core / 1 TB RAM node; all downstream R and Python steps ran on a 24-core / 70 GB RAM server. The table below lists minimum and recommended specifications for each stage.

Stage	Step	Min RAM	Rec. RAM	Min CPU	Rec. CPU
Chapter 0	`cellranger mkref`	16 GB	64 GB	8	16
Chapter 0	`cellranger count`	32 GB	64 GB	16	80
Chapter 1	QC → Harmony → clustering	32 GB	64 GB	8	24
Chapter 1	DoubletFinder	32 GB	64 GB	8	24
Chapter 2	DESeq2 → GO → hdWGCNA	16 GB	64 GB	4	16
Chapter 3	scFates pseudotime	16 GB	32 GB	4	8

Storage footprint

All sizes are measured from the 4-sample *Arabidopsis thaliana* run described in this paper.

Component	Size
Docker image (`matigara/scrnaseq:latest`)	10.4 GB
CellRanger 7.1.0 binary	1.9 GB
Reference genome (TAIR10, `cellranger mkref` output)	1.6 GB
Raw FASTQs (4 samples, R1 + R2)	15 – 25 GB
CellRanger output (4 samples, `--no-bam`)	2.5 GB
Seurat checkpoint objects (`.rds`)	1.4 GB
Pipeline results (plots, tables, PDFs)	2.2 GB
Total	**~35 – 45 GB**

Using `--no-bam` in `cellranger count` eliminates per-sample BAM files (~20–30 GB each), reducing storage requirements by approximately 75 %. Intermediate Seurat `.rds` checkpoints (written after each chapter) can be removed once downstream steps are validated to further reduce disk usage.

Methods

Chapter 0 — CellRanger preprocessing (bash)

CellRanger 7.1.0 is included in the Docker image and is available directly from the container shell.

Step 1 — Download raw sequencing data

Raw FASTQ files are deposited in the CNCB Genome Sequence Archive (GSA) under study accession **CRA010863**. The four runs used in this analysis are:

Run	Sample	Condition	R1 size	R2 size
CRR775252	scDS1a	condition_1	8.6 GB	8.8 GB
CRR775253	scDS1b	condition_1	8.0 GB	8.0 GB
CRR775256	scDS2a	condition_2	8.7 GB	8.1 GB
CRR775257	scDS2b	condition_2	10 GB	9.3 GB

Total download size: ~70 GB.

Files can be downloaded via FTP from the CNCB BIG Data Center. Using `aria2c` with parallel connections reduces download time substantially:

```
# Create a URL list file
cat > urls_cra010863.txt << 'EOF'
ftp://download.big.ac.cn/gsa2/CRA010863/CRR775252/CRR775252_f1.fq.gz
ftp://download.big.ac.cn/gsa2/CRA010863/CRR775252/CRR775252_r2.fq.gz
ftp://download.big.ac.cn/gsa2/CRA010863/CRR775253/CRR775253_f1.fq.gz
ftp://download.big.ac.cn/gsa2/CRA010863/CRR775253/CRR775253_r2.fq.gz
ftp://download.big.ac.cn/gsa2/CRA010863/CRR775256/CRR775256_f1.fq.gz
ftp://download.big.ac.cn/gsa2/CRA010863/CRR775256/CRR775256_r2.fq.gz
ftp://download.big.ac.cn/gsa2/CRA010863/CRR775257/CRR775257_f1.fq.gz
ftp://download.big.ac.cn/gsa2/CRA010863/CRR775257/CRR775257_r2.fq.gz
EOF

# Download with aria2 (5 connections per file, 8 files in parallel)
aria2c -x 5 -j 8 --input-file=urls_cra010863.txt

# Alternatively, with wget (sequential)
# wget -i urls_cra010863.txt
```

Step 2 — Build the genome reference index

The reference index is built once per genome assembly and reused across all samples. For *Arabidopsis thaliana* TAIR10 (Ensembl Plants release 59):

```
# Download genome FASTA and gene annotation
wget https://ftp.ensemblgenomes.ebi.ac.uk/pub/plants/release-59/fasta/\
arabidopsis_thaliana/dna/Arabidopsis_thaliana.TAIR10.dna.toplevel.fa.gz
wget https://ftp.ensemblgenomes.ebi.ac.uk/pub/plants/release-59/gtf/\
arabidopsis_thaliana/Arabidopsis_thaliana.TAIR10.59.gtf.gz

gunzip Arabidopsis_thaliana.TAIR10.dna.toplevel.fa.gz
gunzip Arabidopsis_thaliana.TAIR10.59.gtf.gz

# Build the CellRanger reference (output: Arabidopsis_thaliana_TAIR10/)
cellranger mkref \
  --genome=Arabidopsis_thaliana_TAIR10 \
  --fasta=Arabidopsis_thaliana.TAIR10.dna.toplevel.fa \
  --genes=Arabidopsis_thaliana.TAIR10.59.gtf \
  --nthreads=16
```

The resulting `Arabidopsis_thaliana_TAIR10/` directory (1.6 GB) is passed to `--transcriptome` in Step 2.

Step 3 — Prepare FASTQ files

CellRanger requires files named `{SAMPLE}_S{N}_L001_R{1,2}_001.fastq.gz`. Files downloaded from public repositories (e.g. NCBI SRA / CNCB) must be renamed before alignment:

```
# Example: CNCB accession files → CellRanger naming convention
mv CRR775252_f1.fq.gz  scDS1a_S1_L001_R1_001.fastq.gz
mv CRR775252_r2.fq.gz  scDS1a_S1_L001_R2_001.fastq.gz
mv CRR775253_f1.fq.gz  scDS1b_S1_L001_R1_001.fastq.gz
mv CRR775253_r2.fq.gz  scDS1b_S1_L001_R2_001.fastq.gz
mv CRR775256_f1.fq.gz  scDS2a_S1_L001_R1_001.fastq.gz
mv CRR775256_r2.fq.gz  scDS2a_S1_L001_R2_001.fastq.gz
mv CRR775257_f1.fq.gz  scDS2b_S1_L001_R1_001.fastq.gz
mv CRR775257_r2.fq.gz  scDS2b_S1_L001_R2_001.fastq.gz
```

Read 1 (R1) carries the 10x Chromium cell barcode and UMI (28 bp); Read 2 (R2) carries the cDNA insert.

Step 4 — Run alignment

Each sample is aligned independently. The helper function below checks for a completed output before launching, making the script safely re-runnable:

```
#!/bin/bash
set -euo pipefail

REF=/workspace/Arabidopsis_thaliana_TAIR10   # output of cellranger mkref

run_if_needed() {
  local id="$1"
  if [ -f "${id}/outs/metrics_summary.csv" ]; then
    echo "[$(date '+%F %T')] ${id}: already complete — skipping."
```

```
    return 0
  fi
  echo "[$(date '+%F %T')] Starting ${id} ..."
  cellranger count \
    --id="${id}" \
    --fastqs=. \
    --sample="${id}" \
    --transcriptome="${REF}" \
    --localcores=80 \
    --no-bam
  echo "[$(date '+%F %T')] ${id}: done."
}

run_if_needed scDS1a
run_if_needed scDS1b
run_if_needed scDS2a
run_if_needed scDS2b
```

The `--no-bam` flag suppresses BAM output and reduces per-sample disk usage by ~20–30 GB. The `--localcores` value should be set to the number of available CPU threads.

Step 5 — Output structure

Each completed run produces the following layout:

```
scDS1a/
└── outs/
    ├── filtered_feature_bc_matrix/     ← input to Chapter 1
    │   ├── barcodes.tsv.gz
    │   ├── features.tsv.gz
    │   └── matrix.mtx.gz
    ├── metrics_summary.csv             ← per-sample QC summary
    └── web_summary.html                ← interactive QC report
```

The `filtered_feature_bc_matrix/` directory is read by `load_seurat_samples()` at the start of Chapter 1 (Section 1).

Chapter 1 — Single-cell processing

Running Option 1 opens a self-contained terminal inside the container, separate from the host shell it was launched from:

```
docker run -it --rm \
  -v ~/your-workdir:/workspace \
  matigara/scrnaseq:latest /bin/bash
```

`R` is then started from that terminal, on its own:

```
R
```

Chapter 1 (`capitulo1_single_cell.R`) runs entirely from this session.

Initialization

After entering the R session, Chapter 1 starts by defining the two paths that control the analysis. `PIPELINE_DIR` points to the helper scripts inside the container. `DATA_DIR` is the mounted project directory, and `base_dir` is the root under which all step-specific outputs are written. In a standard Docker run these paths do not need to be changed.

```
PIPELINE_DIR <- "/workspace/ScRNASeq-Docker/workflow"
DATA_DIR   <- "/workspace/."
base_dir   <- file.path(DATA_DIR, "resultados")
```

The analysis environment is then initialized by loading the package set, custom plotting utilities, and core workflow functions. A fixed random seed is used throughout the chapter, Seurat 5 compatibility is enabled, and the working directory is set to the mounted project folder.

```
source(file.path(PIPELINE_DIR, "load_libraries.R"))
source(file.path(PIPELINE_DIR, "custom_seurat.R"))
source(file.path(PIPELINE_DIR, "ScRNA_Analysis_Functions.R"))

set.seed(1807)
options(Seurat.allow.s4 = FALSE)
setwd(DATA_DIR)
```

Finally, the pipeline creates the output directory structure under `resultados/`. The helper functions use `output_dir` as the active destination for plots and tables; it is initialized to `base_dir` and reassigned at the start of later sections.

```
list2env(create_pipeline_dirs(base_dir), envir = .GlobalEnv)
output_dir <- base_dir
```

Section 1 — Data loading and pre-filter QC

Section 1 begins by selecting the first output folder and declaring the sample manifest. Each sample entry provides the CellRanger `filtered_feature_bc_matrix` directory, the label used in figures, and the experimental condition used downstream. The same structure is used for any number of samples; only paths, labels, and condition names need to be adapted.

```
output_dir <- dir_01

samples <- list(
  list(file = "cellranger_v2/scDS1a/outs/filtered_feature_bc_matrix",
       label = "scDS1a", condition = "condition_1"),
  list(file = "cellranger_v2/scDS1b/outs/filtered_feature_bc_matrix",
       label = "scDS1b", condition = "condition_1"),
  list(file = "cellranger_v2/scDS2a/outs/filtered_feature_bc_matrix",
       label = "scDS2a", condition = "condition_2"),
```

```
  list(file = "cellranger_v2/scDS2b/outs/filtered_feature_bc_matrix",
       label = "scDS2b", condition = "condition_2")
)
```

One color is assigned to each sample label so that all QC and UMAP plots use the same visual identity throughout the workflow.

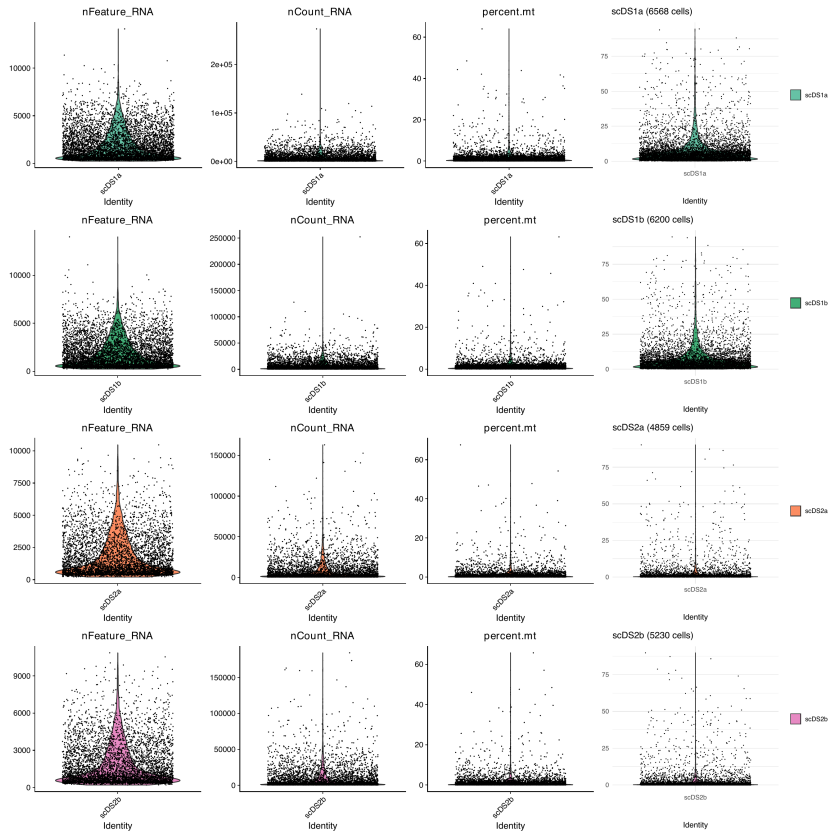
```
colors <- c(
  "scDS1a" = "#66c2a5",
  "scDS1b" = "#41ae76",
  "scDS2a" = "#fc8d62",
  "scDS2b" = "#e34a33"
)
```

The Arabidopsis mitochondrial and chloroplast gene prefixes are then used to compute organelle-derived read fractions for every cell. These pre-filter QC distributions are saved as `qc_prefilter.pdf` and are inspected before choosing the filtering thresholds in the next step.

```
mt_pattern <- "^ATMG"
cp_pattern <- "^ATCG"

seurat_list_raw <- load_seurat_samples(
  samples = samples,
  DATA_DIR = DATA_DIR,
  mt_pattern = mt_pattern,
  cp_pattern = cp_pattern
)

plot_qc_batch(seurat_list_raw, colors, "qc_prefilter.pdf")
```



Representative pre-filter QC output produced by `plot_qc_batch()`.

Section 2 — Cell filtering and doublet detection

Section 2 applies the first cell-level exclusion criteria. The default thresholds retain cells with at least 200 detected genes and less than 5% mitochondrial signal. DoubletFinder is then applied sample-by-sample through the filtering helper, and the same QC panel is regenerated after filtering.

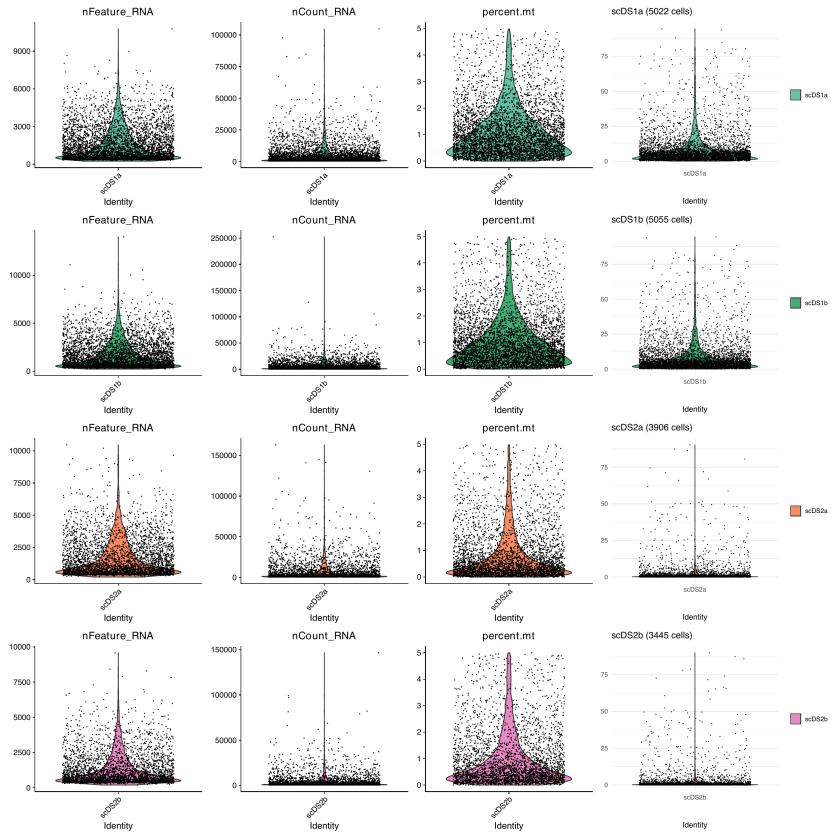
```
output_dir <- dir_01

seurat_list <- filter_seurat_samples(
  seurat_list_raw,
  min_features = 200,
  max_mt = 5
)

plot_qc_batch(seurat_list, colors, "qc_postfilter.pdf")
```

The filtered list is saved as a checkpoint so the workflow can restart from this point without reloading and refiltering the raw matrices.

```
saveRDS(
  seurat_list,
  file.path(dir_objects, "seurat_list_postfilter.rds")
)
```



Representative post-filter QC output after cell filtering and doublet removal.

Section 3 — Merge and initial preprocessing

The filtered samples are merged into one Seurat object and processed with the standard log-normalization workflow. Variable features are selected by VST, the matrix is scaled, PCA is computed over 30 components, and an initial UMAP is generated from the PCA space. This pre-Harmony UMAP is used to visualize sample structure before batch correction.

```
output_dir <- dir_01

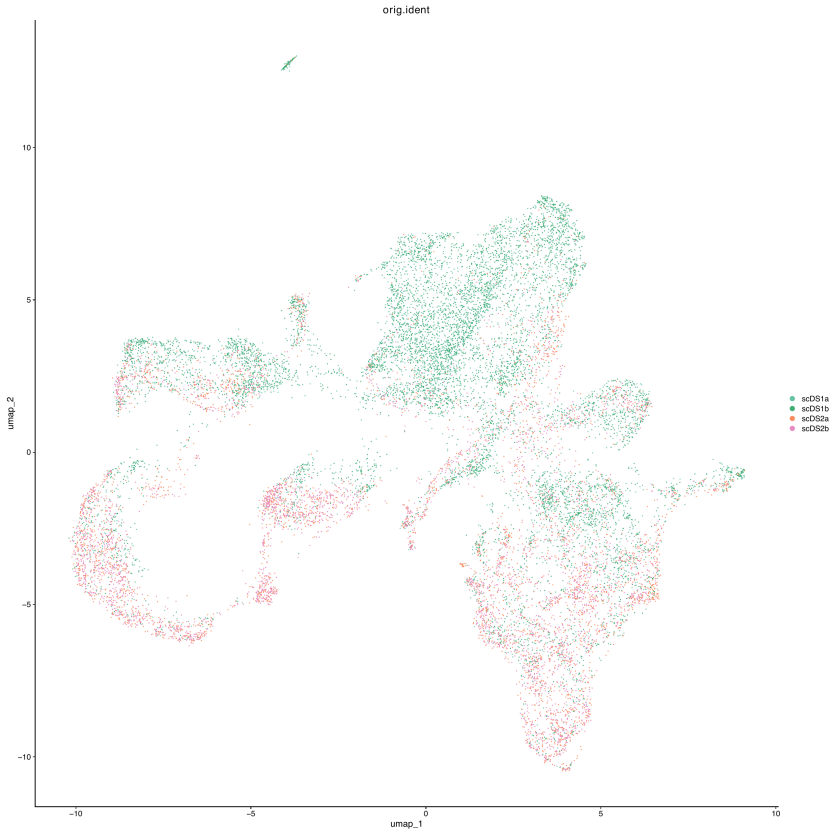
pbmc_harmony <- reduce(seurat_list, merge) %>%
  NormalizeData(verbose = FALSE) %>%
  FindVariableFeatures(
    selection.method = "vst",
    nfeatures = 2000,
    verbose = FALSE
  ) %>%
  ScaleData(verbose = FALSE) %>%
  RunPCA(npcs = 30, verbose = FALSE) %>%
  RunUMAP(reduction = "pca", dims = 1:30, verbose = FALSE)
```

The pre-correction UMAP is saved by sample identity, followed by a checkpoint of the merged object.

```
save_pdf(
  DimPlot(pbmc_harmony, group.by = "orig.ident", cols = colors),
```

```
  "umap_preharmony.pdf"
)

saveRDS(
  pbmc_harmony,
  file.path(dir_objects, "pbmc_harmony_preharmony.rds")
)
```



Representative UMAP before Harmony correction, colored by sample identity.

Section 4 — Harmony batch correction

Harmony is then run on the sample identity field (`orig.ident`) to reduce sample-level batch structure while preserving biological variation. The UMAP embedding is recomputed from the Harmony reduction, and downstream sections use Harmony coordinates rather than the original PCA space.

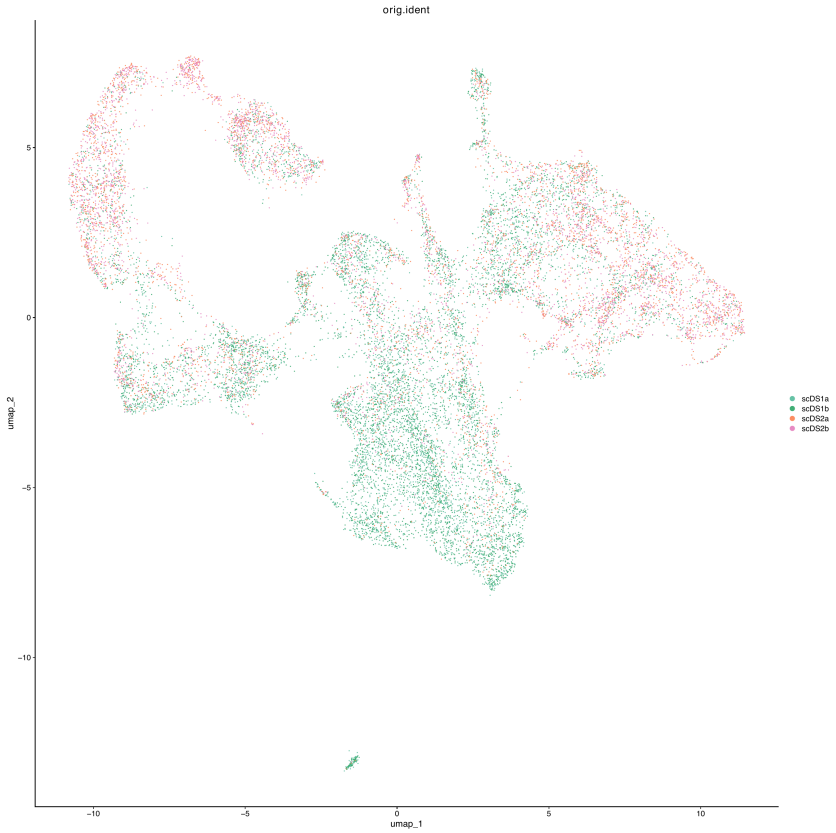
```
output_dir <- dir_01

pbmc_harmony <- pbmc_harmony %>%
  RunHarmony("orig.ident", plot_convergence = FALSE) %>%
  RunUMAP(reduction = "harmony", dims = 1:30, verbose = FALSE)
```

The post-Harmony UMAP and object checkpoint are written to disk for later resolution testing, clustering, annotation, and export.

```
save_pdf(
  DimPlot(pbmc_harmony, group.by = "orig.ident", cols = colors),
  "umap_postharmony.pdf"
)

saveRDS(
  pbmc_harmony,
  file.path(dir_objects, "pbmc_harmony_postharmony.rds")
)
```



Representative UMAP after Harmony correction, colored by sample identity.

Section 5 — Resolution optimization

Cluster granularity is selected with a cluster tree (clustree), which tracks how Leiden communities split or merge across candidate resolutions. The lowest resolution at which clusters stabilize is used in Section 6.

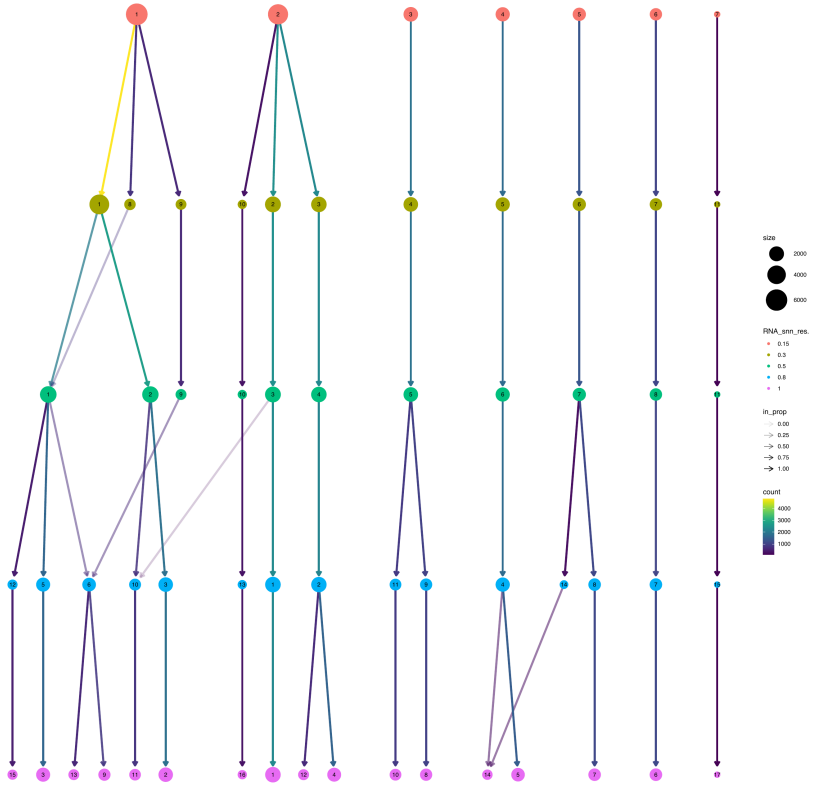
```
resolutions_test <- c(0.15, 0.30, 0.50, 0.8, 1.0)
output_dir <- dir_02

clu <- pbmc_harmony %>%
  RunUMAP(reduction = "harmony", dims = 1:30, verbose = FALSE) %>%
  FindNeighbors(
    reduction = "harmony",
    dims = 1:30,
    k.param = 30,
    verbose = FALSE
```

```
  )

for (res in resolutions_test) {
  clu <- FindClusters(clu, resolution = res, algorithm = 4, verbose = FALSE)
}

save_pdf(clustree(clu, prefix = "RNA_snn_res."), "clustree.pdf", w = 18, h = 18)
```



Cluster tree across candidate Leiden resolutions.

Section 6 — Final clustering

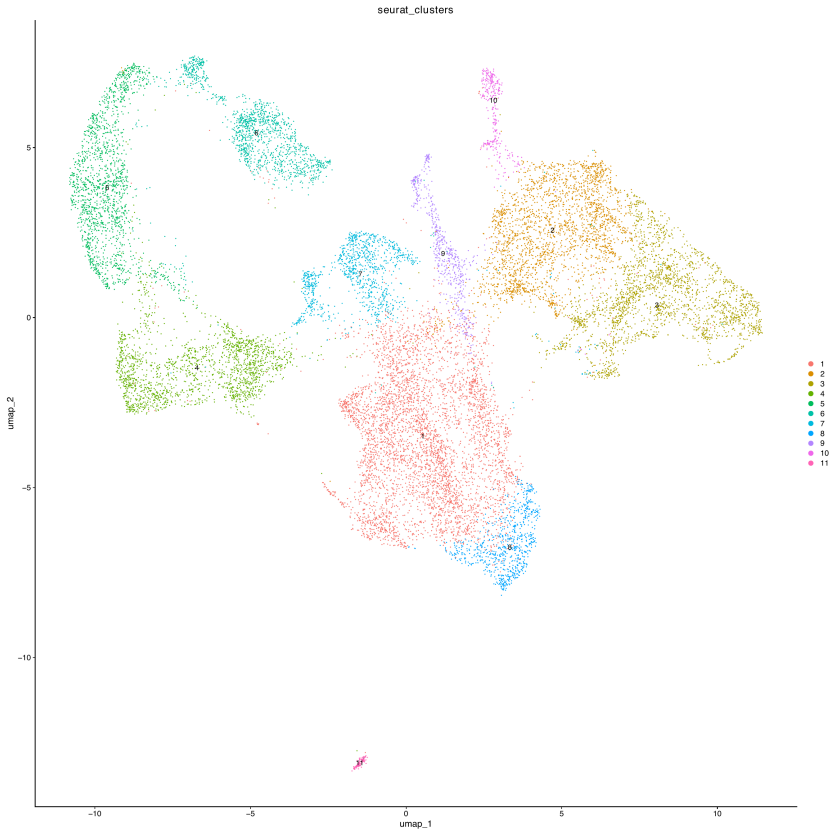
The selected resolution is applied to the Harmony graph to obtain the final cluster assignment. The neighbor graph is rebuilt from the Harmony embedding, clustered with the Leiden algorithm, and the resulting clusters are stored as the active identity class.

```
cluster_resolution <- 0.8
output_dir <- dir_02

pbmc_harmony <- pbmc_harmony %>%
  RunUMAP(reduction = "harmony", dims = 1:30, verbose = FALSE) %>%
  FindNeighbors(
    reduction = "harmony",
    dims = 1:30,
    k.param = 30,
    verbose = FALSE
  ) %>%
```

```
  FindClusters(resolution = cluster_resolution, algorithm = 4, verbose = FALSE)

Idents(pbmc_harmony) <- "seurat_clusters"
save_pdf(
  DimPlot(pbmc_harmony, group.by = "seurat_clusters", label = TRUE),
  "umap_seuratclusters.pdf"
)
```



Final clustering at resolution 0.8, labeled by Seurat cluster.

Section 7 — Cell-type annotation

Two complementary strategies assign cell-type labels to clusters: a bibliography-based approach that crosses cluster-level differential genes with the marker table, and a reference-transfer approach that projects labels from a published Arabidopsis leaf atlas (GSE273033).

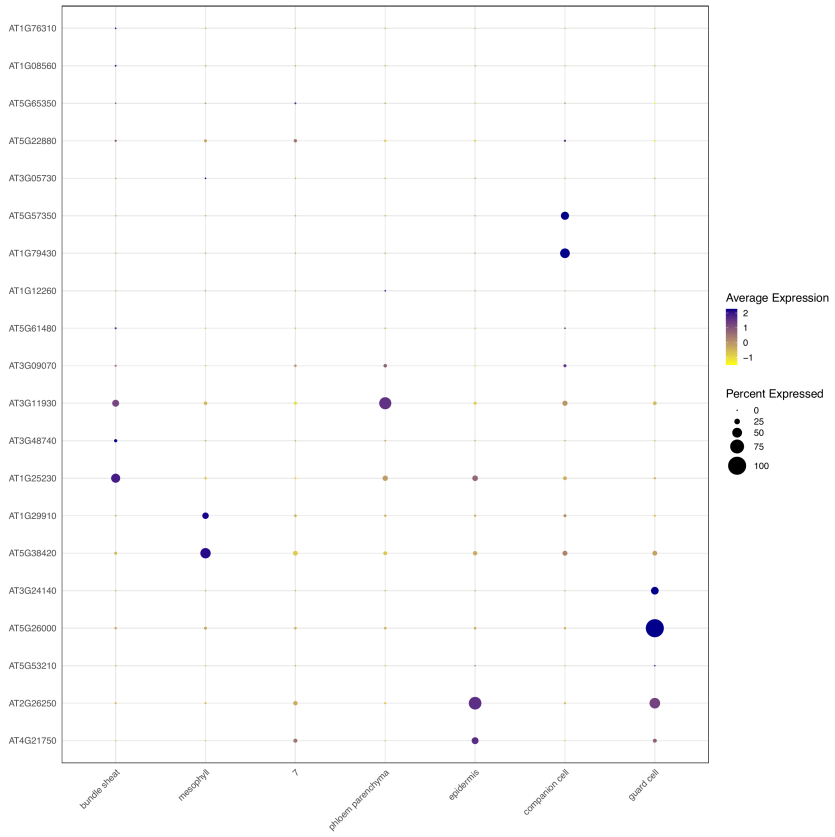
The bibliography-based method assigns each cluster the cell type whose marker gene is ranked highest (lowest adjusted p-value) among that cluster's differential genes. When multiple clusters independently match the same cell type, they receive unique suffixed labels (`Epidermis.1`, `Epidermis.2`, etc.) so that each original cluster is preserved as a distinct entity in all downstream analyses.

```
markers <- find_markers(
  pbmc_harmony,
  output_file = file.path(output_dir, "FindAllMarkers.tsv")
)

pbmc_harmony <- annotate_by_markers(
```

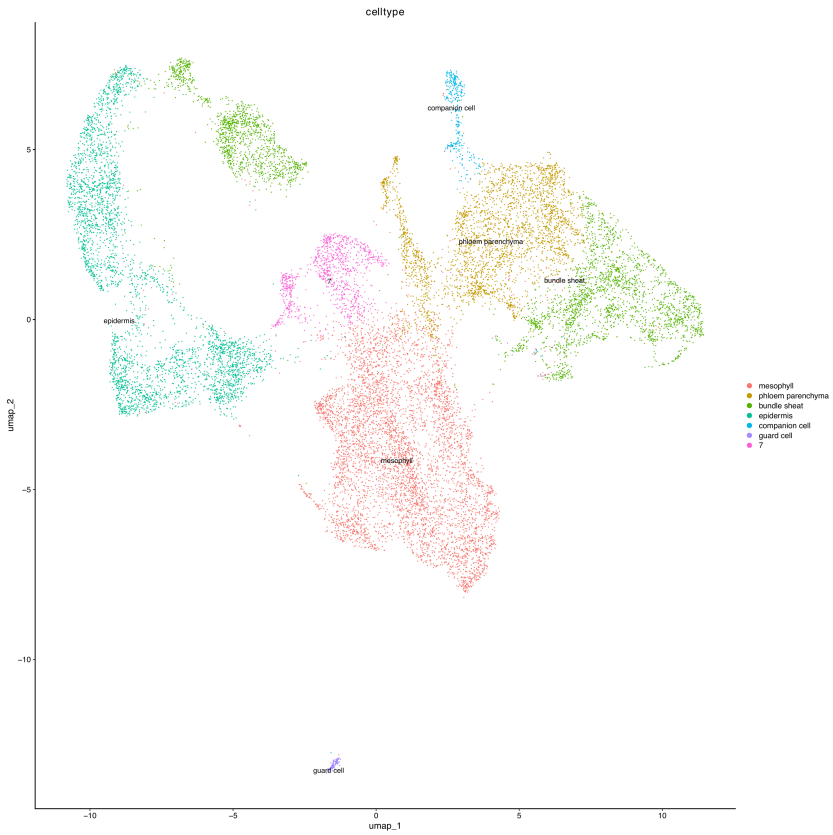
```
  pbmc_harmony,
  markers,
  reference_file = biblio_marks_file
)

plot_marker_dotplot(
  pbmc_harmony,
  marker_table,
  annot_col = "celltype",
  outfile = file.path(output_dir, "dotplot_marker_table_annotation_biblio.pdf"),
  width = 18, height = 18
)
```



Marker dotplot after bibliography-based cell-type assignment.

```
save_pdf(
  DimPlot(
    pbmc_harmony,
    group.by = "celltype",
    label = TRUE,
    repel = TRUE,
    raster = FALSE
  ),
  "umap_annotation_biblio.pdf"
)
```



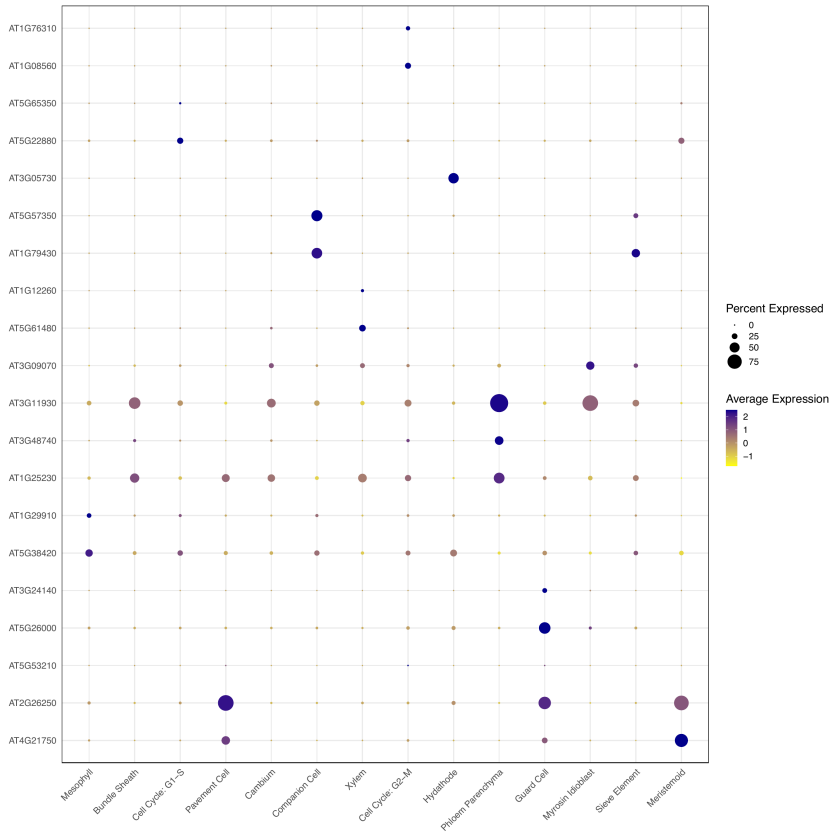
UMAP colored by bibliography-based cell-type annotation.

The reference-transfer strategy projects labels from the published atlas onto the dataset using `FindTransferAnchors`/`TransferData`, storing the result in `pbmc_harmony$celltype_reference`.

```
reference_obj <- readRDS(
  file.path(DATA_DIR, "GSE273033_seuratObj_for_publication.rds")
)

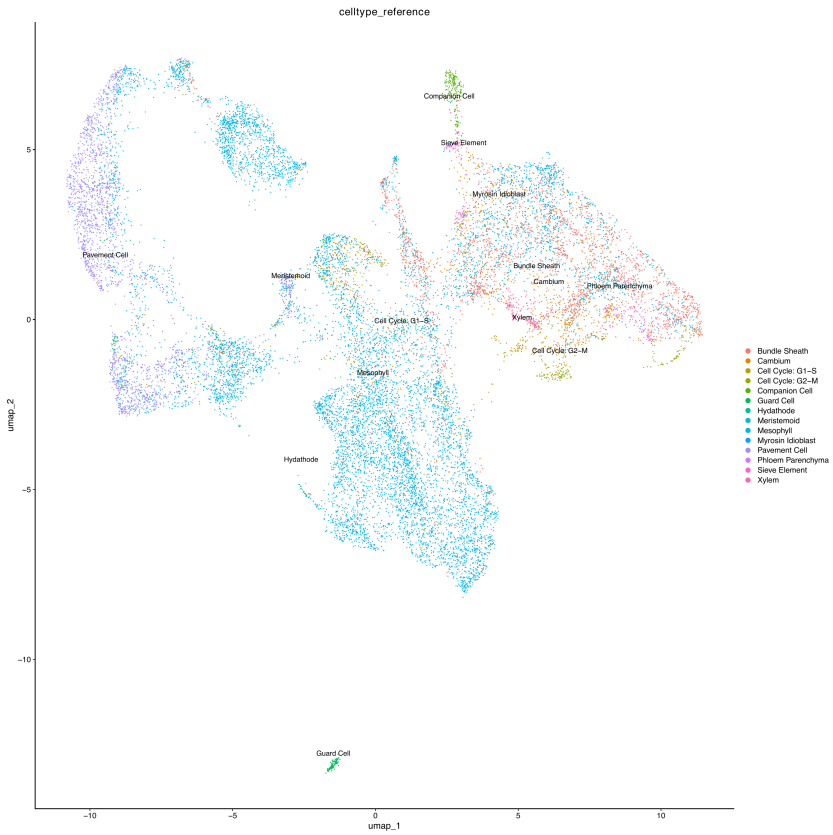
pbmc_harmony <- annotate_by_reference(
  pbmc_harmony,
  reference_obj = reference_obj,
  reference_col = "annotation"
)

plot_marker_dotplot(
  pbmc_harmony,
  marker_table,
  annot_col = "celltype_reference",
  outfile = file.path(output_dir, "dotplot_marker_table_annotation_reference.pdf"),
  width = 18, height = 18
)
```



Marker dotplot after reference-transfer cell-type assignment.

```
save_pdf(
  DimPlot(
    pbmc_harmony,
    group.by = "celltype_reference",
    label = TRUE,
    repel = TRUE,
    raster = FALSE
  ),
  "umap_annotation_reference.pdf"
)
```



UMAP colored by reference-transfer cell-type annotation.

Section 9 — Annotated cluster tree

The resolution sweep from Section 5 is repeated with each node labeled by the most frequent reference-transfer cell type it contains, confirming that the chosen resolution cleanly separates known cell types.

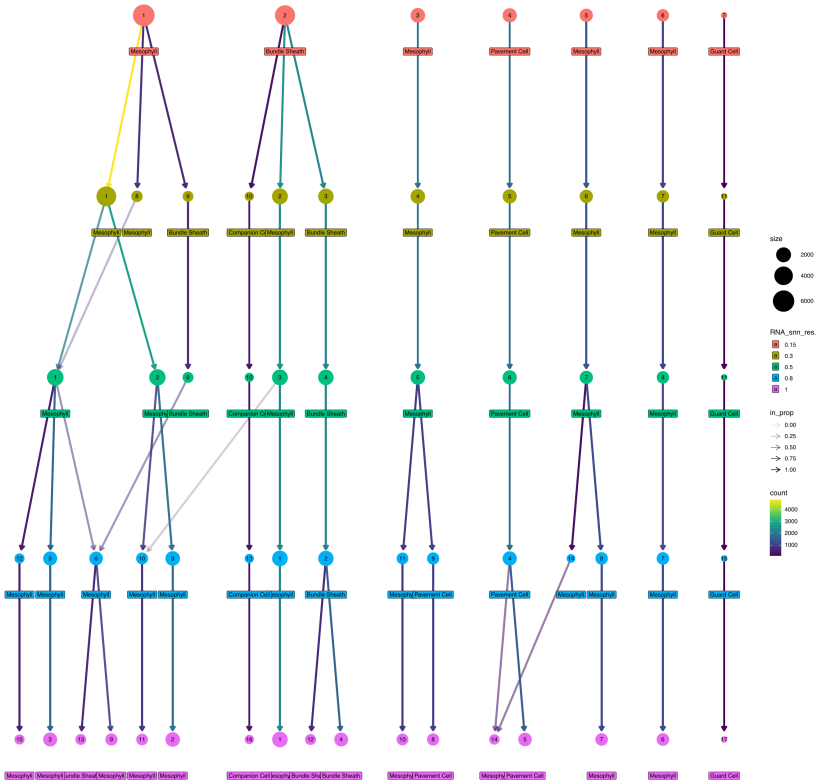
```
Mode <- function(x) {
  ux <- unique(x)
  ux[which.max(tabulate(match(x, ux)))]
}

clu <- pbmc_harmony %>%
  RunUMAP(reduction = "harmony", dims = 1:30, verbose = FALSE) %>%
  FindNeighbors(
    reduction = "harmony",
    dims = 1:30,
    k.param = 30,
    verbose = FALSE
  )

for (res in resolutions_test) {
  clu <- FindClusters(clu, resolution = res, algorithm = 4, verbose = FALSE)
}

save_pdf(
  clustree(
    clu,
```

```
    prefix = "RNA_snn_res.",
    node_label = "celltype_reference",
    node_label_aggr = "Mode"
  ),
  "clustree_annotated.pdf", w = 18, h = 18
)
```



Cluster tree across candidate resolutions, labeled by reference-transfer cell type.

Section 10 — Gene expression visualization

Violin and feature plots are generated for a single gene and for a small gene set, both across all annotated cell types and within one cell type of interest. `JoinLayers()` is required in Seurat 5 before subsetting an object built from a merge.

```
gene <- "AT5G26000"
genes_of_interest <- c("AT5G26000", "AT5G54250")
celltype <- "Guard Cell"

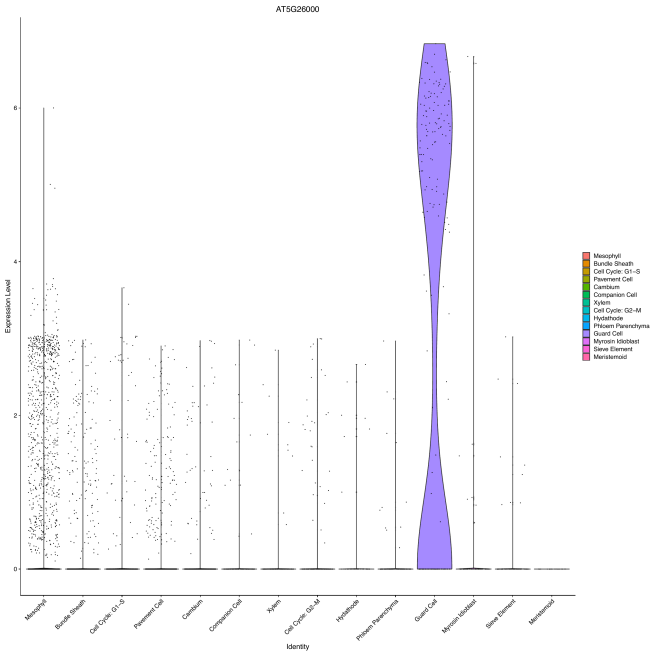
output_dir <- dir_04

pbmc_harmony <- JoinLayers(pbmc_harmony)
Idents(pbmc_harmony) <- "celltype_reference"

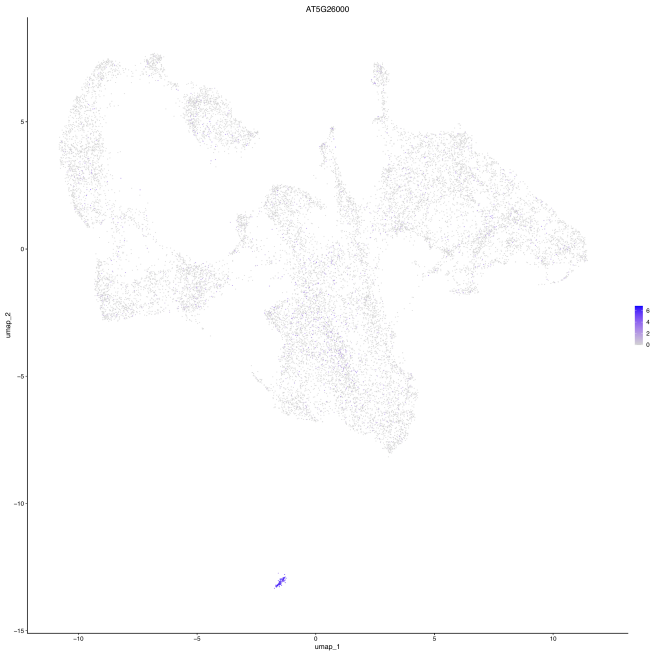
n_genes <- length(genes_of_interest)

save_vln(VlnPlot(pbmc_harmony, features = gene), "vln_gene_all.pdf")
```

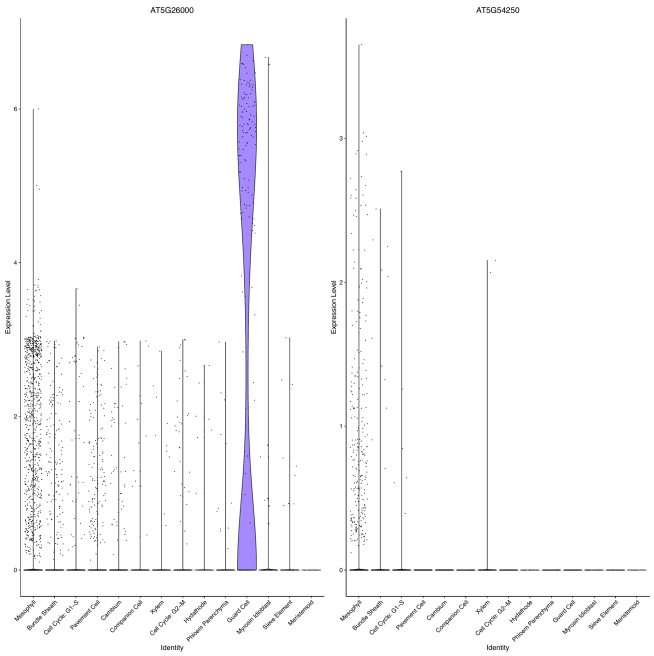
```r
save_pdf(FeaturePlot(pbmc_harmony, features = gene), "feature_gene_all.pdf")
save_vln(
  VlnPlot(pbmc_harmony, features = genes_of_interest),
  "vln_geneset_all.pdf",
  n = n_genes
)
save_pdf(
  FeaturePlot(pbmc_harmony, features = genes_of_interest),
  "feature_geneset_all.pdf",
  w = 18, h = 18
)
```



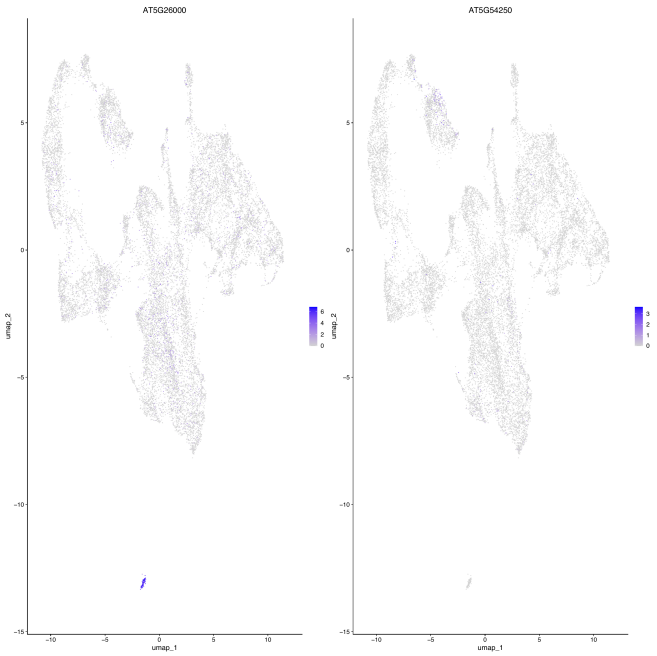
Violin, single gene



Feature, single gene



Violin, gene set

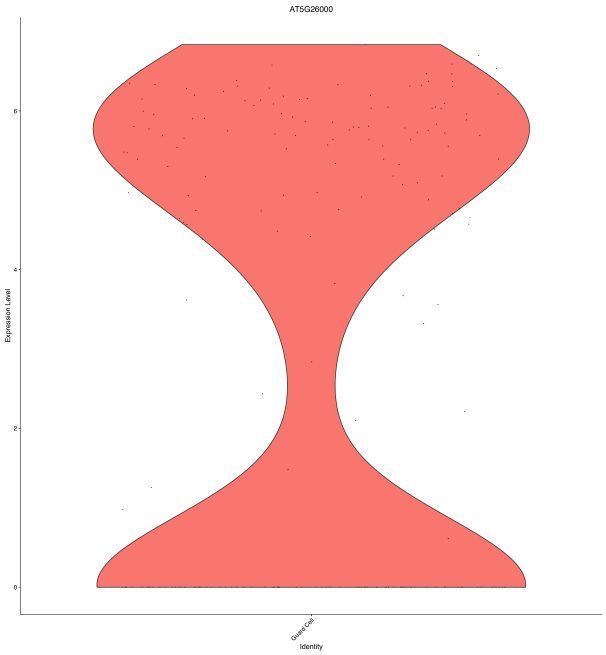


Feature, gene set

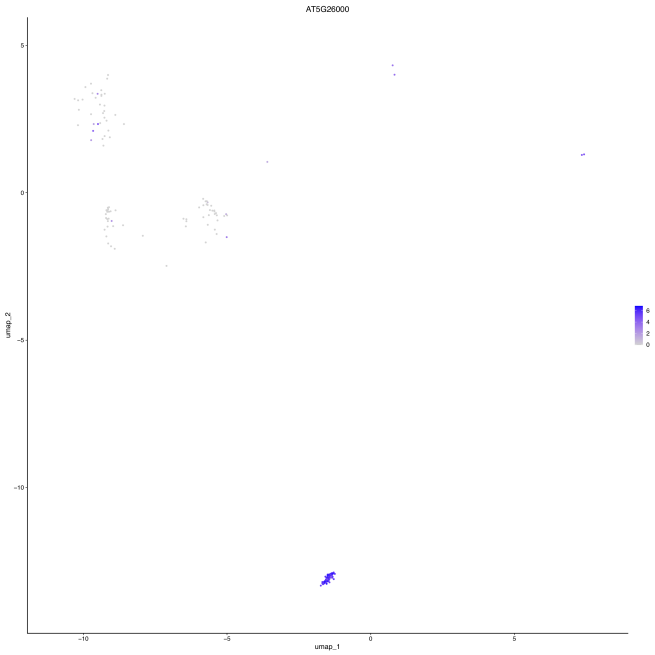
Expression across all annotated cell types.

```
sub_obj <- subset(pbmc_harmony, idents = celltype)

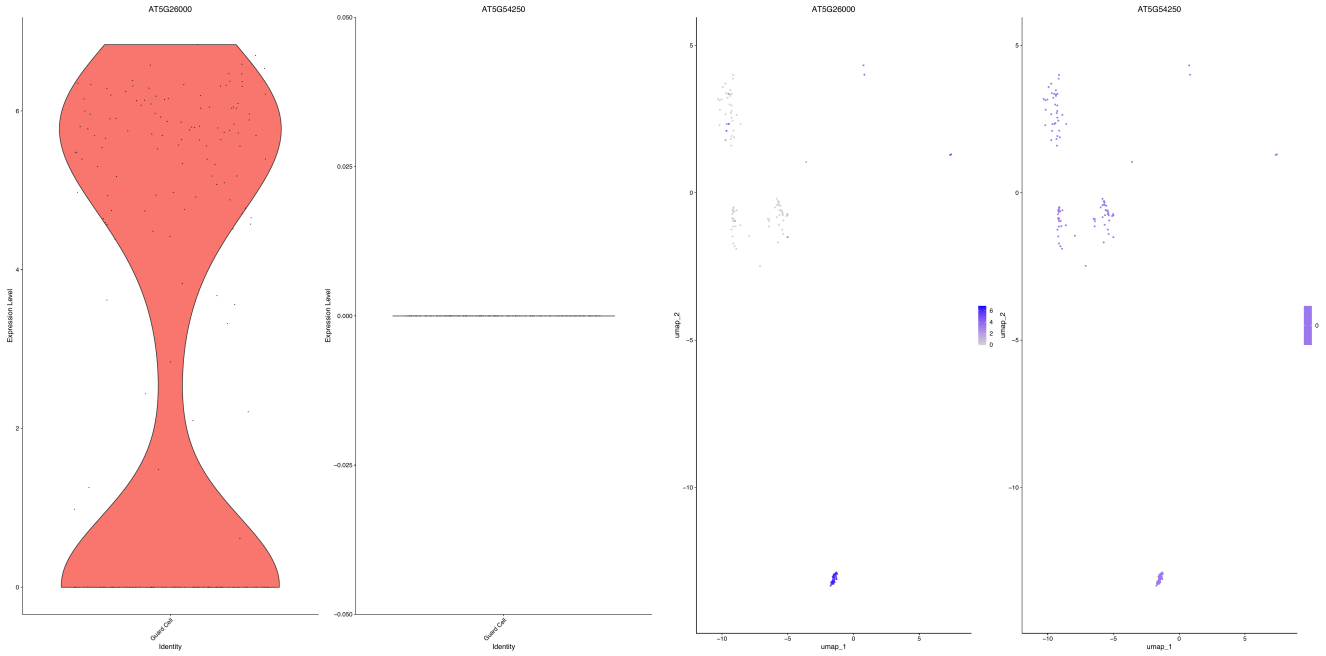
save_vln(VlnPlot(sub_obj, features = gene), "vln_gene_celltype.pdf")
save_pdf(FeaturePlot(sub_obj, features = gene), "feature_gene_celltype.pdf")
save_vln(
  VlnPlot(sub_obj, features = genes_of_interest),
  "vln_geneset_celltype.pdf",
  n = n_genes
)
save_pdf(
  FeaturePlot(sub_obj, features = genes_of_interest),
  "feature_geneset_celltype.pdf",
  w = 18, h = 18
)
```



Violin, single gene



Feature, single gene



Violin, gene set

Feature, gene set

Expression restricted to the cell type of interest (Guard Cell).

Section 11 — Cell-type grouping (optional)

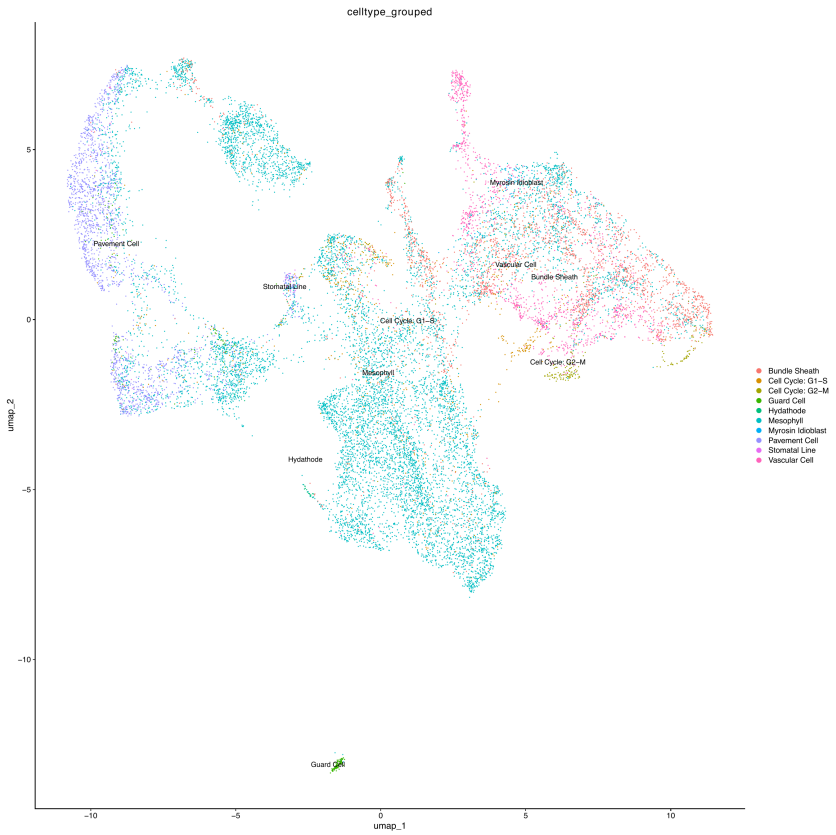
Fine-grained labels not needed for downstream analysis are collapsed into broader categories; any cell type not listed in `grouping` keeps its original label. This section can be skipped if no merging is required.

```
grouping <- c(
  "Companion Cell" = "Vascular Cell",
  "Cambium" = "Vascular Cell",
  "Phloem Parenchyma" = "Vascular Cell",
  "Xylem" = "Vascular Cell",
  "Sieve Element" = "Vascular Cell",
  "Meristemoid" = "Stomatal Line"
)

output_dir <- dir_05

pbmc_harmony$celltype_grouped <- recode(
  pbmc_harmony$celltype_reference, !!!grouping
)

save_pdf(
  DimPlot(
    pbmc_harmony,
    group.by = "celltype_grouped",
    label = TRUE, repel = TRUE, raster = FALSE
  ),
  "umap_grouped.pdf"
)
```



UMAP after collapsing fine-grained labels into broader groups.

Section 12 — Interactive cell-type curation (optional)

Populations that appear heterogeneous in the UMAP are subclustered, inspected against the marker table, and manually reassigned where needed. This section must be run interactively, step by step, rather than sourced as a whole; it can be skipped if the Section 8 annotation is already satisfactory. Below, `Mesophyll` and `Pavement Cell` are subclustered and re-inspected with `plot_subcluster_umap()`. In addition to these individual UMAPs, `save_subcluster_composite()` (not shown) writes a multi-page inspection PDF: its first page places every cell type's subcluster UMAP side by side, and each following page shows, for one cell type at a time, a FeaturePlot of every marker gene in the bibliography table. This composite is what the researcher actually reviews to decide, per subcluster, which marker genes are expressed and therefore which final cell-type label it should receive — those decisions are then encoded in the reassignment table below. Every resulting subcluster here confirms its parent label, so the reassignment is effectively a no-op in this generic example, but the same workflow is how a cluster mixing two cell types would be split and relabeled correctly.

```
curation_col <- "celltype_grouped"
Idents(pbmc_harmony) <- curation_col

mesophyll_umap     <- subcluster_cell_type(pbmc_harmony, "Mesophyll", annot_col =
        curation_col)
pavement_cell_umap <- subcluster_cell_type(pbmc_harmony, "Pavement Cell", annot_col =
        curation_col)

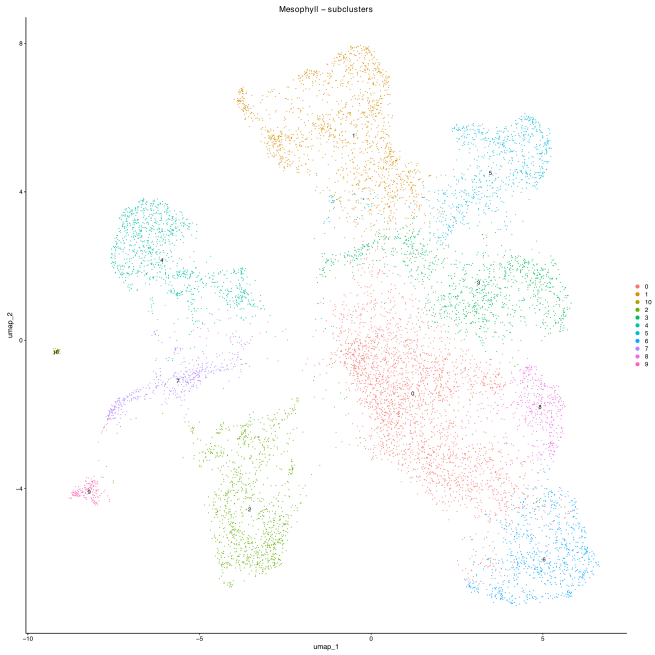
p_meso_dim <- plot_subcluster_umap(mesophyll_umap, "Mesophyll", output_dir)
```

```r
p_pave_dim <- plot_subcluster_umap(pavement_cell_umap, "Pavement Cell", output_dir)

reassign <- list(
  mesophyll_umap     = c("0" = "Mesophyll", "1" = "Mesophyll", "2" = "Mesophyll",
        "others" = "Mesophyll"),
  pavement_cell_umap = c("0" = "Pavement Cell", "1" = "Pavement Cell", "2" =
        "Pavement Cell",
                         "3" = "Pavement Cell", "4" = "Pavement Cell", "others" =
        "Pavement Cell")
)

pbmc_harmony <- apply_subcluster_reassignment(
  pbmc_harmony,
  subcluster_list = list(mesophyll_umap = mesophyll_umap, pavement_cell_umap =
        pavement_cell_umap),
  reassign = reassign, source_col = curation_col, dest_col = "celltype_curated"
)

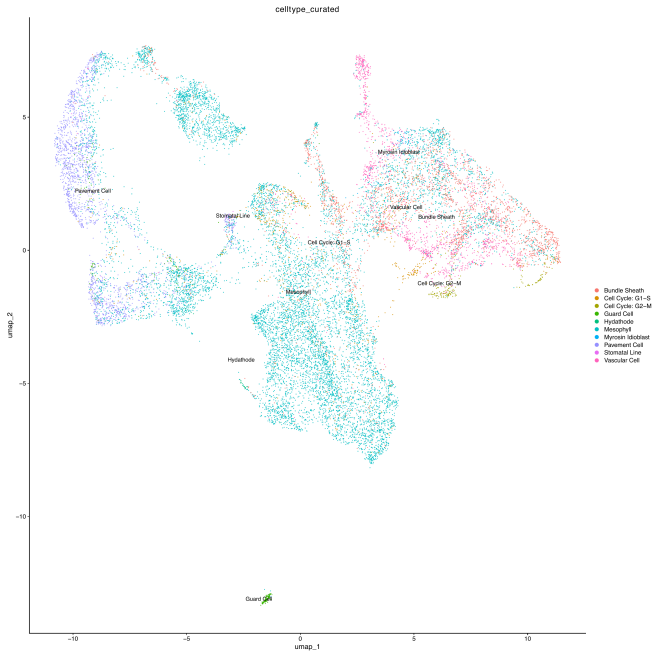
save_pdf(
  DimPlot(pbmc_harmony, group.by = "celltype_curated", label = TRUE, repel = TRUE,
        raster = FALSE),
  "umap_curated.pdf"
)
```



Mesophyll subclusters



Pavement Cell subclusters



Final curated UMAP

Subcluster inspection and final result after reassignment.

Section 13 — Export to h5ad (Scanpy / Python)

The curated object is exported to AnnData `h5ad` format for the Python-based trajectory and velocity analyses covered in later chapters (Scanpy, scFates, Palantir — all pre-installed in the Docker image). `export_to_scanpy()` cleans non-serializable slots, converts the object to a `SingleCellExperiment`, and writes counts, log-normalized data, and the PCA, UMAP, and Harmony reductions into a single file.

```
export_to_scanpy(
  pbmc_harmony,
  file.path(dir_objects, "pbmc_harmony_curated.h5ad")
)
```

This closes Chapter 1; `pbmc_harmony_curated.rds` and `pbmc_harmony_curated.h5ad` are the two checkpoints consumed downstream, by Chapter 2 (pseudobulk differential expression) and Chapter 3 (pseudotime), respectively.

Section 14 — Cell-type subsets

Chapter 2 reloads the curated object and splits it into one Seurat subset per curated cell type, which pseudobulk aggregation operates on independently. Subset names are sanitized so they double safely as list names and output filenames.

```
pbmc_harmony <- readRDS(file.path(dir_objects, "pbmc_harmony_curated.rds"))

pseudobulk_annot_col <- "celltype_curated"
```

```
cell_type_subsets <- create_cell_type_subsets(
  pbmc_harmony, annot_col = pseudobulk_annot_col
)
```

Section 15 — Pseudo-replicate assignment

DESeq2 requires replicates, but pseudobulk samples are aggregated from single cells with no biological replication built in. This step randomly partitions the cells of each condition, within each cell type, into pseudo-replicates; only cell types with at least two conditions are kept for Chapter 2.

```
pseudobulk_conditions <- NULL
n_pseudoreps <- 3

cell_type_subsets_replicates <- assign_pseudoreplicates_batch(
  cell_type_subsets,
  pseudobulk_conditions = pseudobulk_conditions,
  n_pseudoreps = n_pseudoreps
)
```

Section 16 — Pseudobulk tables and DESeq2

Counts are aggregated by pseudo-replicate within each cell type, and DESeq2 runs the contrasts listed in `comparisons` — here `condition_1` vs. `condition_2`, the two example conditions defined back in Section 1. `log2FC > 0` means higher expression in the second condition of the pair.

```
comparisons <- list(
  list(conds = c("condition_1", "condition_2"), tag = "condition_1_vs_condition_2")
)

cell_types_to_analyze <- NULL
output_dir <- dir_06

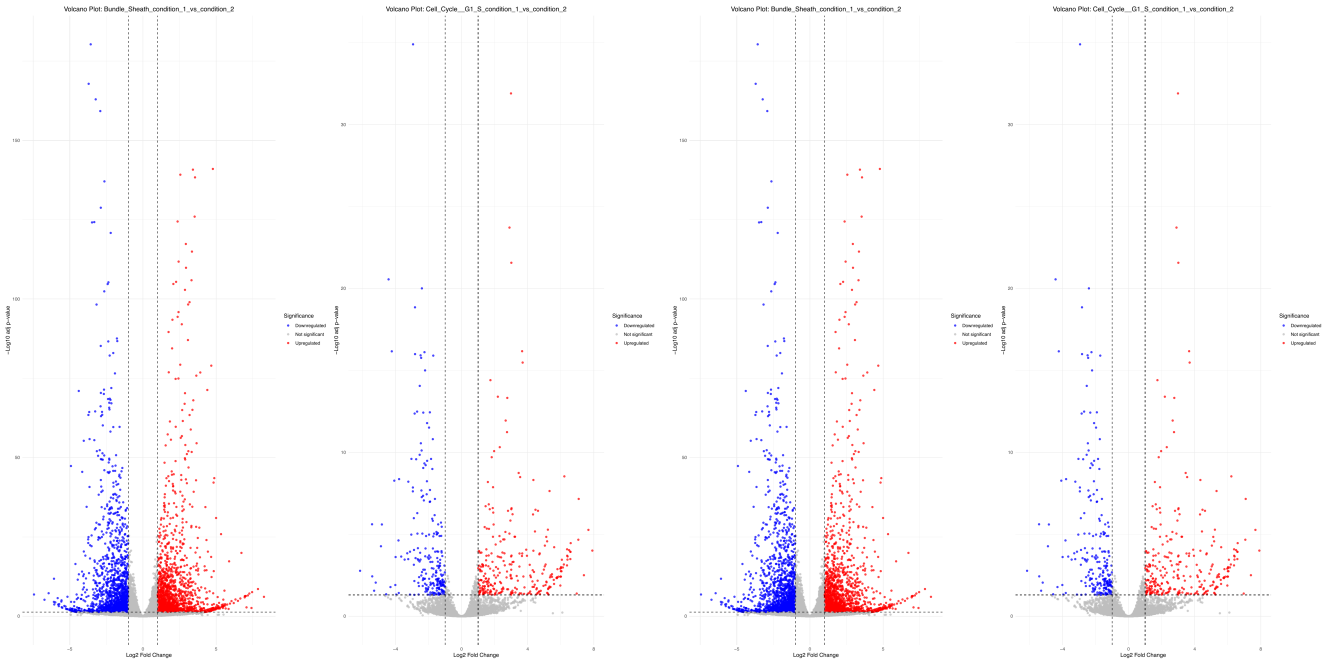
deseq2_results <- run_pseudobulk_deseq2_analysis(
  cell_type_subsets_replicates = cell_type_subsets_replicates,
  comparisons = comparisons,
  output_dir = output_dir,
  cell_types = cell_types_to_analyze,
  pseudobulk_dir = file.path(dir_objects, "pseudobulk_replicas")
)
```

Section 17 — Volcano plots

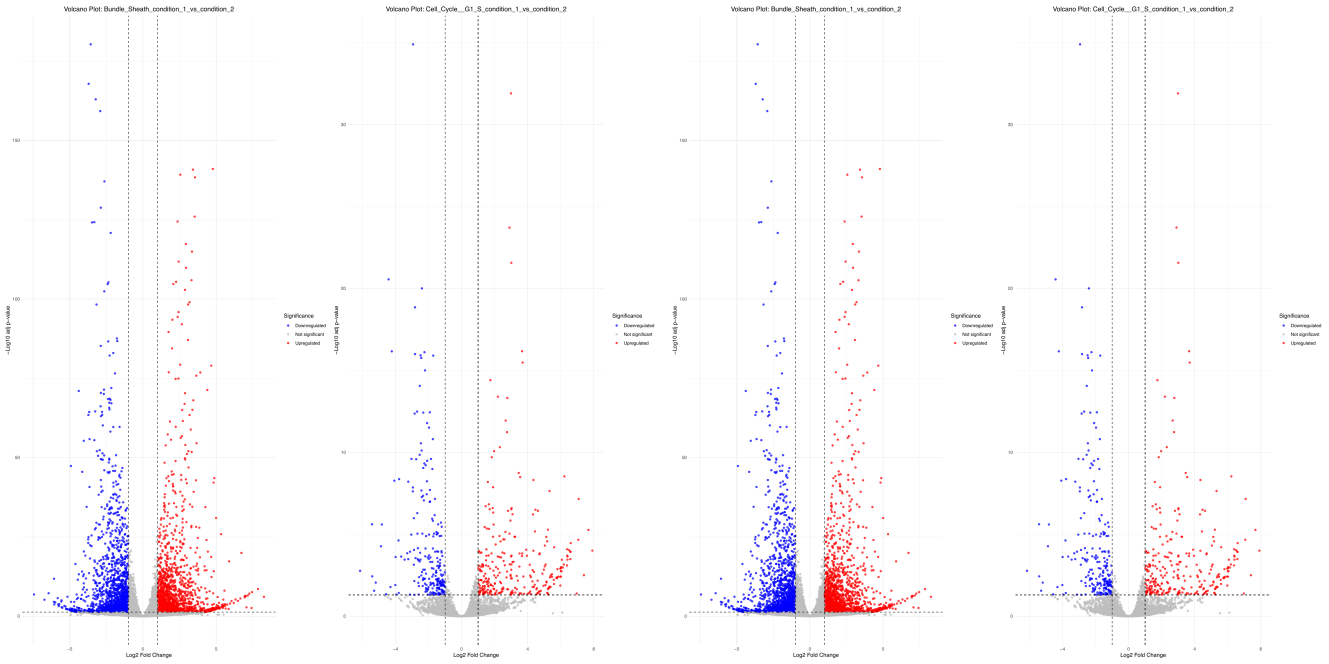
`render_volcano_plots()` renders one PNG volcano plot per cell type for the selected contrast, and also combines all of them into a single multi-page PDF. Four representative cell types are shown below; the other six follow the same layout.

```
volcano_tag <- "condition_1_vs_condition_2"
padj_cut <- 0.05
lfc_cut <- 1
```

```
render_volcano_plots(
  results_dir = file.path(dir_06, volcano_tag),
  output_dir = file.path(dir_06, volcano_tag, "volcano"),
  pdf_name = paste0("VolcanoPlots_", volcano_tag, ".pdf"),
  padj_cut = padj_cut,
  lfc_cut = lfc_cut
)
```



Guard Cell Mesophyll



Pavement Cell Bundle Sheath

condition_1 vs. condition_2, four of the ten analyzed cell types.

Section 18 — Differential gene tables

`build_differential_tables()` reads every per-cell-type DESeq2 CSV for the contrast, writes a filtered table per cell type, and assembles two combined matrices across all cell types: a discrete classification matrix (-1/0/1 for down/not-significant/up) and the corresponding log2FC matrix. Only genes significant in at least one cell type are kept. In this generic run, 7552 genes pass the filter across the ten cell types — from 4442 in Mesophyll down to a single gene in Stomatal Line, reflecting how much each subset's cell count limits detection power.

```
diff_prefix <- paste0("diff_table_", volcano_tag)

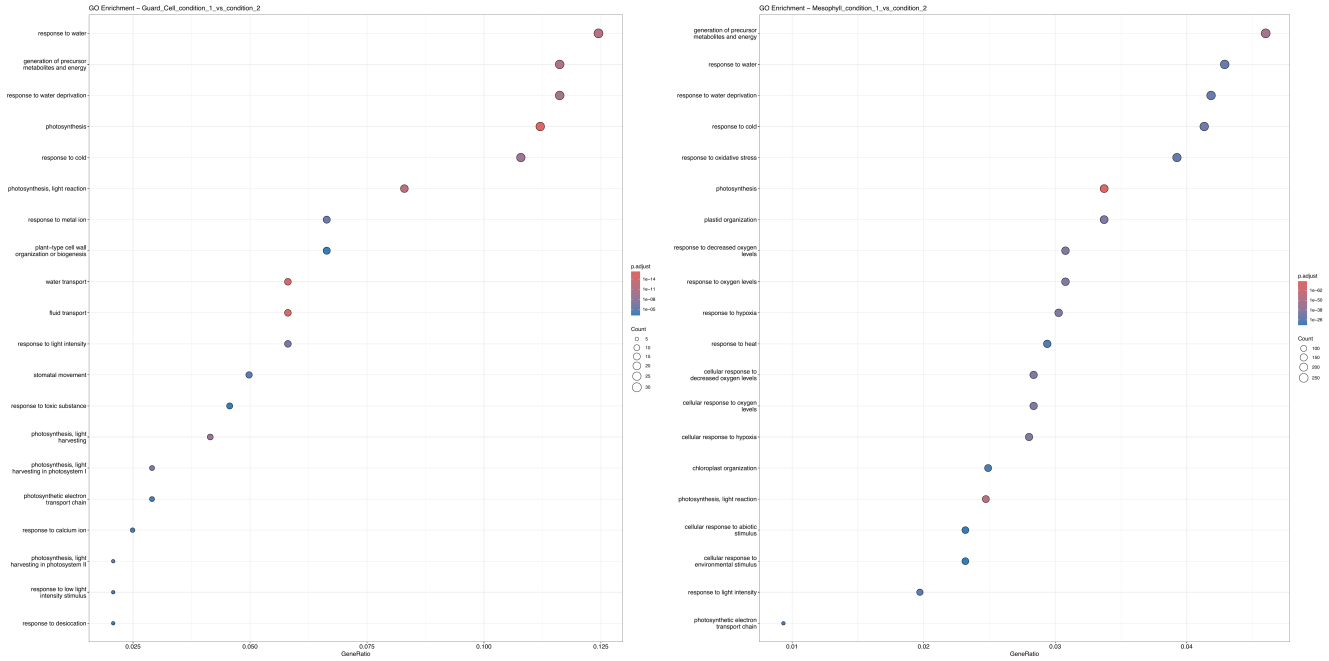
diff_tables <- build_differential_tables(
  results_dir = file.path(dir_06, volcano_tag),
  output_dir = file.path(dir_06, volcano_tag),
  padj_cut = padj_cut,
  lfc_cut = lfc_cut,
  prefix = diff_prefix
)
```

Section 19 — GO enrichment (simple)

`run_go_enrichment_for_contrast()` runs Gene Ontology enrichment per cell type on the same significant-gene sets, against the Arabidopsis annotation (`org.At.tair.db`, keytype `"TAIR"`). Guard Cell, used as the running example throughout this chapter, enriches for "stomatal movement" and several photosynthesis/water-response terms — biologically consistent with its identity. Cell types with too few (or zero) significant genes, such as Stomatal Line above, are skipped with a "no gene can be mapped" message rather than failing the run.

```
go_space <- "BP"
go_orgdb <- org.At.tair.db
go_keytype <- "TAIR"
padj_cutoff <- 0.05

go_results <- run_go_enrichment_for_contrast(
  results_dir = file.path(dir_06, volcano_tag),
  output_dir = file.path(dir_07, volcano_tag),
  orgdb = go_orgdb,
  keytype = go_keytype,
  go_space = go_space,
  padj_cutoff = padj_cutoff,
  contrast_tag = volcano_tag
)
```



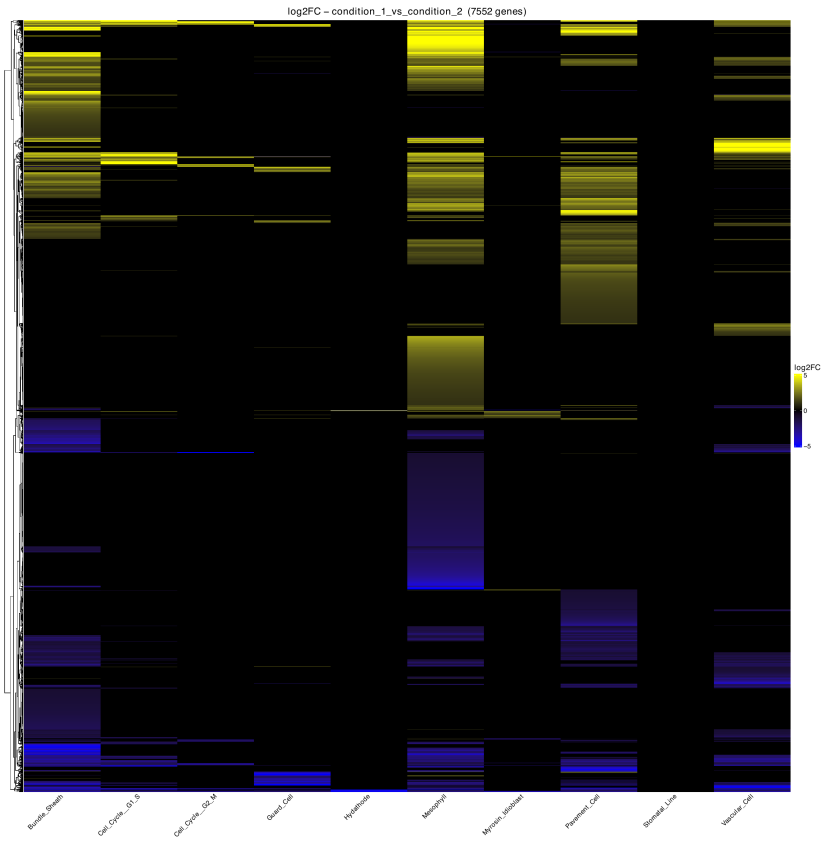
Guard Cell Mesophyll

Section 20 — Log2FC heatmap

A single heatmap summarizes log2FC across all cell types for the contrast, with genes ordered by hierarchical clustering. The blank Stomatal Line column reflects the same lack of significant genes noted in Section 18.

```
heatmap_limits <- c(-5, 5)

build_logfc_heatmap(
  logfc_table = diff_tables$logfc,
  contrast_tag = volcano_tag,
  output_dir = file.path(dir_06, volcano_tag),
  limits = heatmap_limits
)
```



log2FC across all cell types, clustered by gene similarity.

Section 21 — Co-expression network (hdWGCNA)

A weighted co-expression network (Topological Overlap Matrix, TOM) is built from the complete significant-gene set produced in Section 20. Metacells are aggregated within each cell-type × sample combination to reduce single-cell sparsity before network construction. The soft-thresholding power is selected automatically as the lowest value whose scale-free topology fit reaches $R^2 \geq 0.8$; if no power in the tested range reaches that threshold the fallback is 6. `min_module_size` and `deep_split` control the granularity of the internal hierarchical clustering used to construct the TOM.

```
wgcna_name      <- "unified"
n_metacells     <- 50
soft_power      <- NULL
min_module_size <- 20
deep_split      <- 2

run_unified_hdwgcna(
  seurat_obj      = pbmc_harmony,
  de_table_path   = file.path(dir_06, volcano_tag, "tabla_log2FC_fc1_padj_005.tsv"),
  output_dir      = dir_08,
  annot_col       = pseudobulk_annot_col,
  sample_col      = "orig.ident",
  wgcna_name      = wgcna_name,
  n_metacells     = n_metacells,
  soft_power      = soft_power,
  min_module_size = min_module_size,
```

```
  deep_split      = deep_split
)
```

Section 22 — TF co-expression network

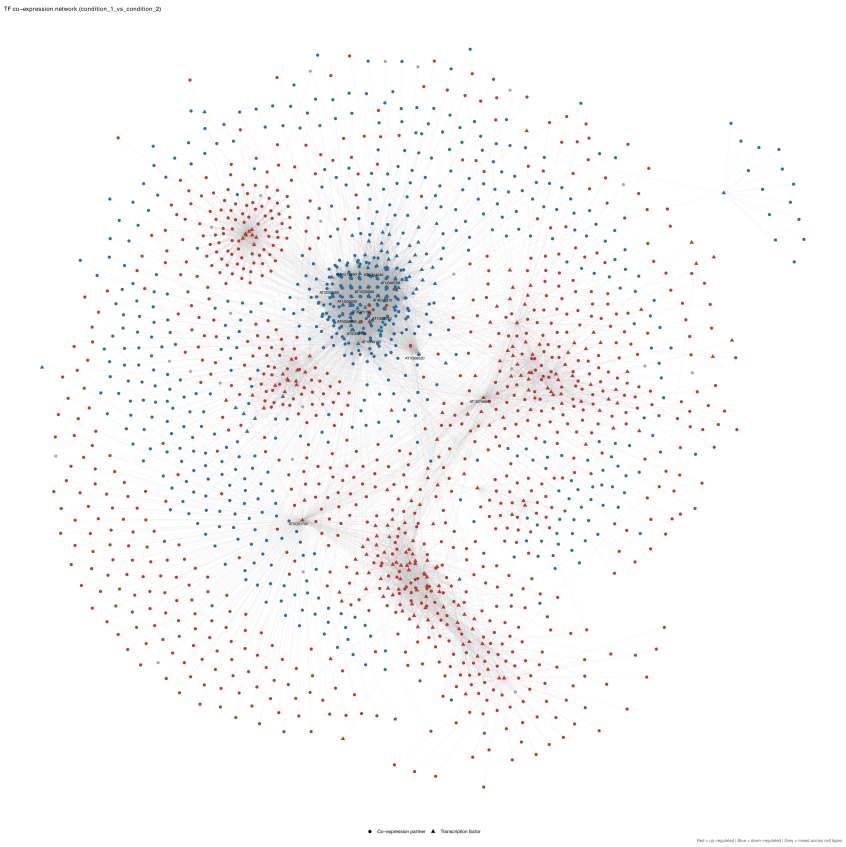
The TOM produced in Section 21 is filtered to a biologically meaningful edge set by retaining only pairs where at least one gene is a known Arabidopsis transcription factor, dropping all target–target edges. The TF locus list comes from AtTFDB (1,840 loci across 50 families), part of AGRIS — the Arabidopsis Gene Regulatory Information Server — freely downloadable from agris-knowledgebase.org/Downloads. Nodes are colored by their predominant direction of differential expression in the contrast of interest: **up** (red) if the gene is significantly up-regulated in all cell types where it is detected, **down** (blue) if down-regulated in all, and **mixed** (grey) if its direction differs across cell types. TFs are drawn as triangles; co-expression partners as circles. The fifteen most highly connected TFs (by degree) are labeled.

Three files are required; their formats are as follows:

File	Format
`edges_unified.tsv`	Tab-separated; columns `source`, `target`, `weight` (TOM ≥ 0.2). 305,394 edges.
`AtTFDB_loci.txt`	Plain text; one TAIR locus per line (e.g. `AT1G01010`). 2,089 entries (AtTFDB + Ath_TF_list, downloaded from agris-knowledgebase.org/AtTFDB).
`tabla_log2FC_fc1_padj_005.tsv`	Gene IDs as row names; one column per cell-type contrast; values = log2FC (`NA` = not significant).

```
edges  <- read.table(file.path(dir_08, "edges_unified.tsv"),
                     header=TRUE, sep="\t")
tfs    <- trimws(readLines(file.path(DATA_DIR, "AtTFDB_loci.txt")))
de_mat <- read.table(
  file.path(dir_06, volcano_tag, "tabla_log2FC_fc1_padj_005.tsv"),
  header=TRUE, sep="\t", row.names=1, check.names=FALSE
)

edges_tf <- edges[edges$weight >= tom_threshold_tf, ]
net_tf   <- build_tf_network(edges_tf, tfs, de_mat)
plot_tf_de_network(net_tf, output_dir = dir_08,
                   layout       = "graphopt",
                   n_hub_label  = n_hub_label_tf,
                   contrast_tag = volcano_tag)
```



TF co-expression network (TOM ≥ 0.2, AtTFDB filter). Triangles = TFs; circles = co-expression partners. Red = up-regulated, blue = down-regulated, grey = mixed direction across cell types.

This closes Chapter 2. The next chapter (pseudotime trajectory analysis) continues from `pbmc_harmony_curated.h5ad` in Python/Scanpy.

Section 24 — Setup

Chapter 3 switches language: the curated object is handed off to Python, and pseudotime trajectories are built with Scanpy, scFates, and Palantir. Library imports and helper functions are loaded the same way the R chapters load theirs — by executing the helper files directly.

```
import os

PIPELINE_DIR = "/workspace/SingleCell/workflow"
DATA_DIR = "/workspace/."
base_dir = os.path.join(DATA_DIR, "resultados")

exec(open(os.path.join(PIPELINE_DIR, "load_libraries_python.py")).read(), globals())
exec(open(os.path.join(PIPELINE_DIR, "ScRNA_Pseudotime_Functions.py")).read(),
        globals())

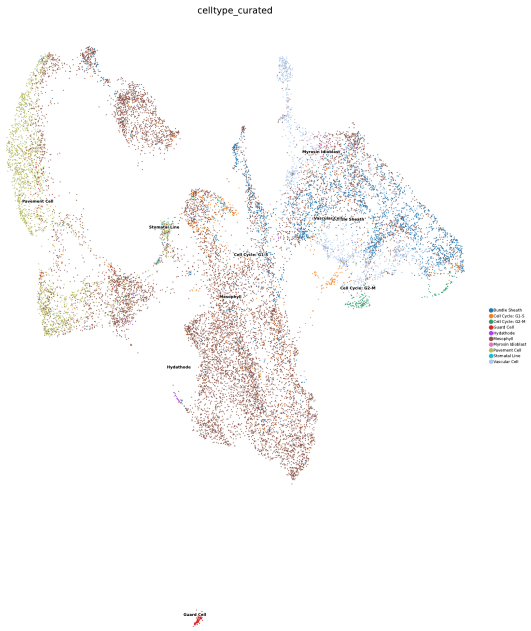
dir_pseudotime = os.path.join(base_dir, "09_pseudotime")
os.makedirs(dir_pseudotime, exist_ok=True)
```

Section 25 — Load curated object

The exported `.h5ad` is read, Seurat-style reduction names are converted to their Scanpy equivalents, and the available cell types are printed for Section 26. An overview UMAP colored by `celltype_curated` is saved for reference.

```
INPUT_H5AD = os.path.join(base_dir, "objects", "pbmc_harmony_curated.h5ad")
ANNOTATION_COL = "celltype_curated"
N_JOBS = 4

adata, N_JOBS = load_curated_object(
    input_h5ad=INPUT_H5AD,
    dir_pseudotime=dir_pseudotime,
    annotation_col=ANNOTATION_COL,
    n_jobs=N_JOBS,
)
```



Overview UMAP, the starting point for trajectory cell-type selection.

Section 26 — Cell type selection

The cell types that make up the trajectory are chosen by name from the Section 25 list. `Guard Cell` and `Stomatal Line` were selected to model the stomatal differentiation axis — from precursor lineage cells to fully differentiated guard cells.

```
TRAJECTORY_CLUSTERS = ["Guard Cell", "Stomatal Line"]

adata_sub = preview_trajectory_selection(
    adata=adata,
    clusters=TRAJECTORY_CLUSTERS,
    annotation_col=ANNOTATION_COL,
    dir_pseudotime=dir_pseudotime,
)
```



Selected subset: Guard Cell and Stomatal Line.

Section 27 — Trajectory inference

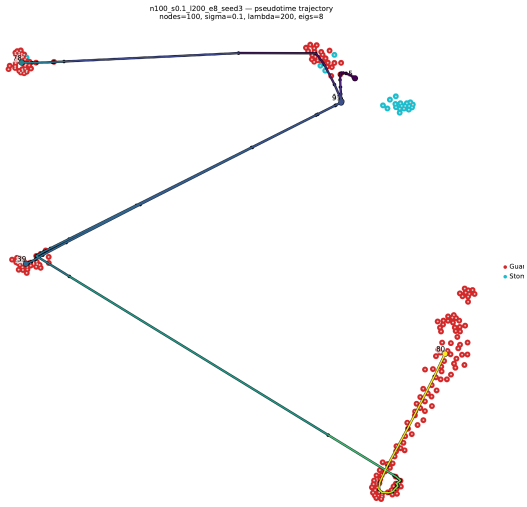
`trajectory_run()` packages one parameter set; `run_trajectory_runs()` builds the principal tree (scFates) for each set and roots it at the chosen cluster. The parameters below — 100 nodes, sigma 0.1, lambda 200, 8 diffusion-map eigenvectors — are the canonical configuration used throughout this project for the real dataset's pseudotime chapter, applied unchanged to the generic example.

```
ROOT_CLUSTER = "Stomatal Line"

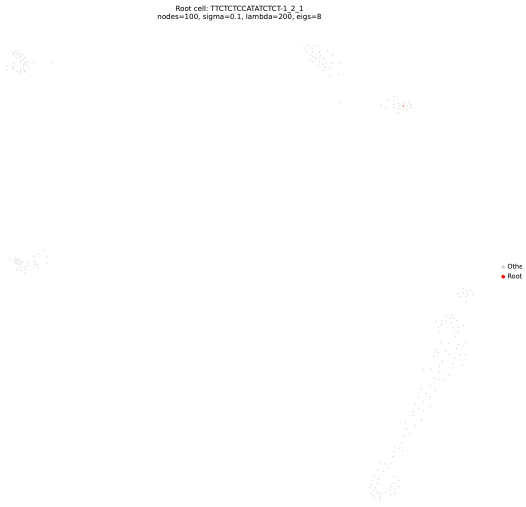
TRAJECTORY_RUNS = [
    trajectory_run(nodes=100, sigma=0.1, lambda_value=200, eigs=8, seed=3),
]

adata_traj, selected_trajectory_dir, trajectory_runs = run_trajectory_runs(
    adata=adata,
```

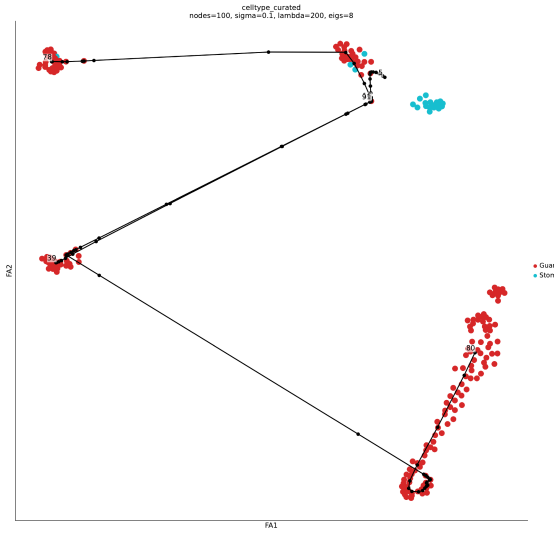
```
    clusters=TRAJECTORY_CLUSTERS,
    root_cluster=ROOT_CLUSTER,
    annotation_col=ANNOTATION_COL,
    output_base_dir=dir_pseudotime,
    runs=TRAJECTORY_RUNS,
)
```



Trajectory, colored by pseudotime



Selected root cell



Tree over cell-type labels

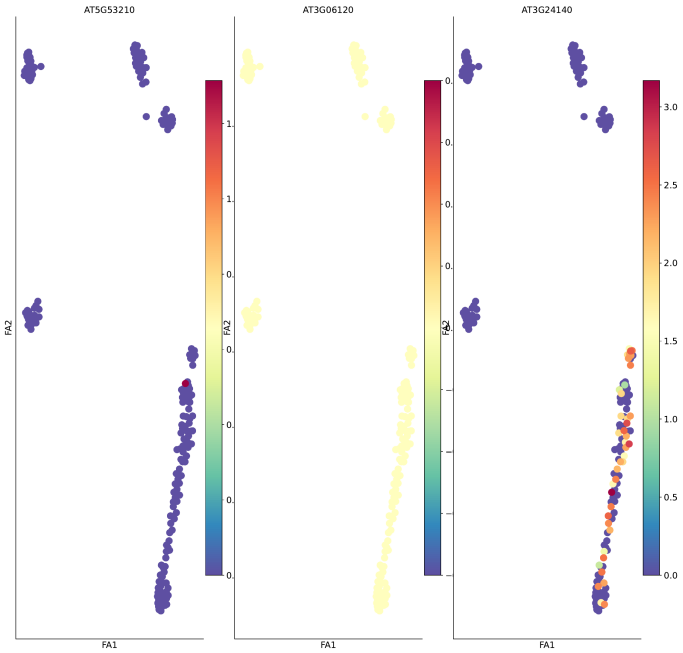
Principal tree for n100_s0.1_l200_e8_seed3, Guard Cell + Stomatal Line, rooted in Stomatal Line.

Section 28 — Plot genes on trajectory

A gene of interest is projected onto the trajectory's force-atlas layout to inspect its expression pattern along the path. `color` accepts either a single gene or a list, producing one panel per gene; one gene is shown here as an example.

```
GENES = ["AT5G53210", "AT3G06120", "AT3G24140"]
name  = os.path.basename(selected_trajectory_dir)

fig = sc.pl.draw_graph(
    adata_traj,
    color=GENES,
    title=GENES,
    show=False,
    return_fig=True,
)
```



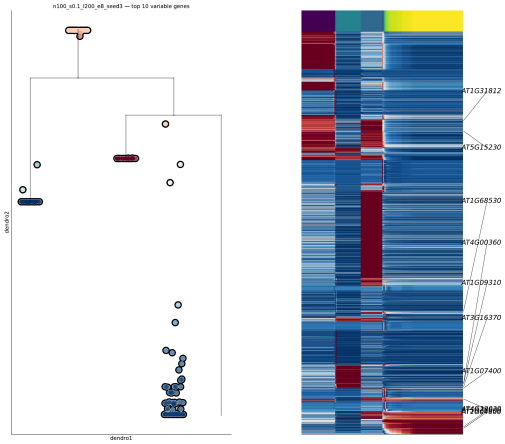
Expression of AT5G53210, AT3G06120, and AT3G24140 along the stomatal guard cell trajectory.

Section 29 — Gene trends

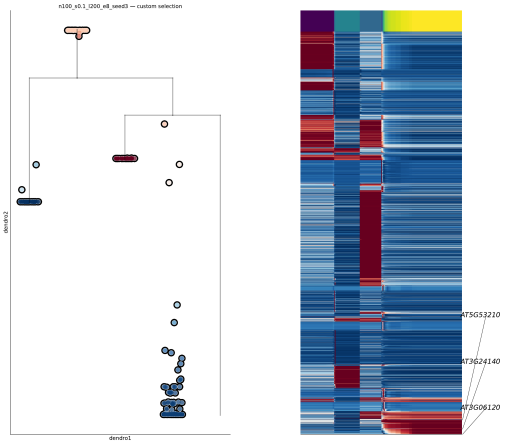
`scf.tl.test_association()` fits a per-segment GAM (`exp ~ s(t, k=spline_df)`) for every gene against pseudotime (22,462 genes tested), then `scf.tl.fit()` smooths the significant hits for two output heatmaps: the top variable genes ordered by their peak along the trajectory, and a custom selection of genes specified by the user.

```
TOP_N = 10
HIGHLIGHT_GENES = ["AT5G53210", "AT3G06120", "AT3G24140"]
ORDERING = "max"
SPLINE_DF = 3

top_path, highlight_path = run_step29_gene_trends(
    adata=adata_traj,
    run_dir=selected_trajectory_dir,
    name=os.path.basename(selected_trajectory_dir),
    custom_genes=HIGHLIGHT_GENES,
    top_n=TOP_N,
    ordering=ORDERING,
    n_jobs=N_JOBS,
    spline_df=SPLINE_DF,
)
```



Top variable genes along the Guard Cell trajectory.



Highlighted genes: AT5G53210, AT3G06120, AT3G24140.

Supplementary Table S1 — Software and package versions

All analyses were executed inside a reproducible Docker container (`matigara/scrnaseq:latest`) based on Ubuntu 24.04.4 LTS (Noble Numbat). The container bundles R 4.5.3 and Python 3.12.3 together with the packages listed below.

R packages

Package	Version
Seurat	5.5.0
SeuratObject	5.4.0
SeuratDisk	0.0.0.9021

Package	Version
harmony	2.0.3
hdWGCNA	0.4.11
DESeq2	1.50.2
clusterProfiler	4.18.4
org.At.tair.db	3.22.0
DoubletFinder	2.0.6
clustree	0.5.1
scater	1.38.1
SingleCellExperiment	1.32.0
zellkonverter	1.20.1
ggplot2	4.0.3
ggrepel	0.9.8
ggraph	2.2.2
patchwork	1.3.2
cowplot	1.2.0
ggpubr	0.6.3
dplyr	1.2.1
tidyr	1.3.2
tidyverse	2.0.0
igraph	2.3.2
tidygraph	1.3.1
WGCNA	1.74
dynamicTreeCut	1.63.1
enrichR	3.4
UCell	2.14.0
GeneOverlap	1.46.0
Matrix	1.7.5
hdf5r	1.3.12
reticulate	1.46.0
BiocGenerics	0.56.0
GenomicRanges	1.62.1

Python packages

Package	Version
scanpy	1.12.1
anndata	0.12.16
scFates	1.2.4
palantir	1.4.4
numpy	2.4.6
pandas	2.3.3
scipy	1.17.1
matplotlib	3.10.9
seaborn	0.13.2
scikit-learn	1.9.0
umap-learn	0.5.12
igraph	1.0.0

This closes Chapter 3 and the methods covered so far.